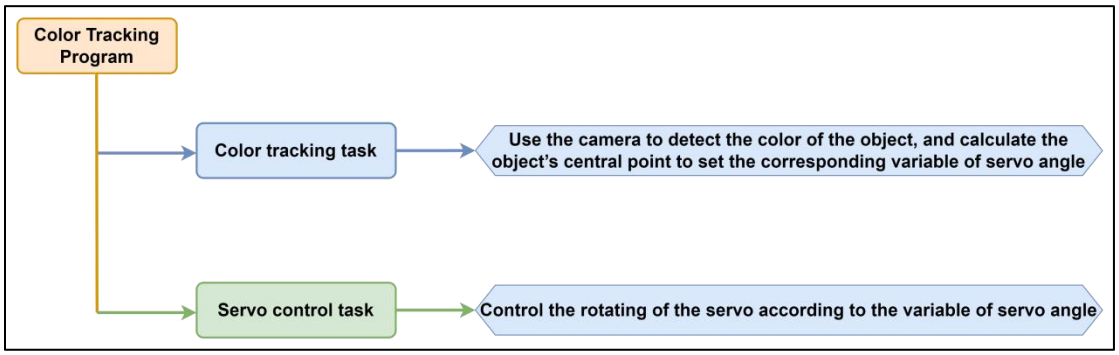
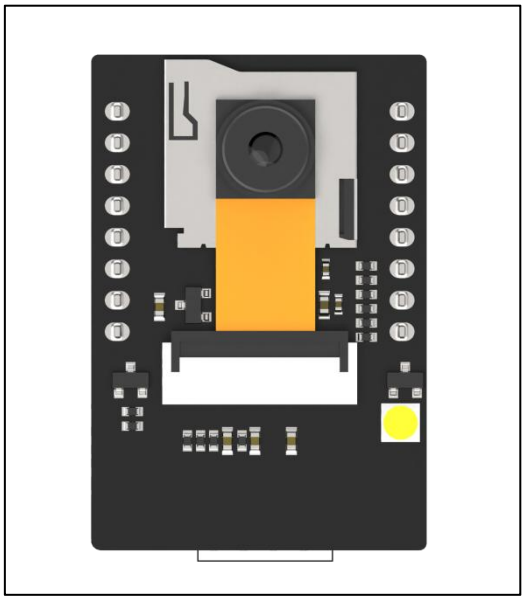
6.5 Color Tracking

In this section, MiniArm will use ESP32-Cam to recognize a blue block, and simultaneously the pan-tile of the MiniArm tracks its movement, turning left and right accordingly.

1. Program Flowchart



2. ESP32-Cam Development Board



ESP32-Cam development board integrates an ESP32 chip and a camera module. After being inserted into the expansion board, it uses the I2C communication interface to read color and face data through I2C

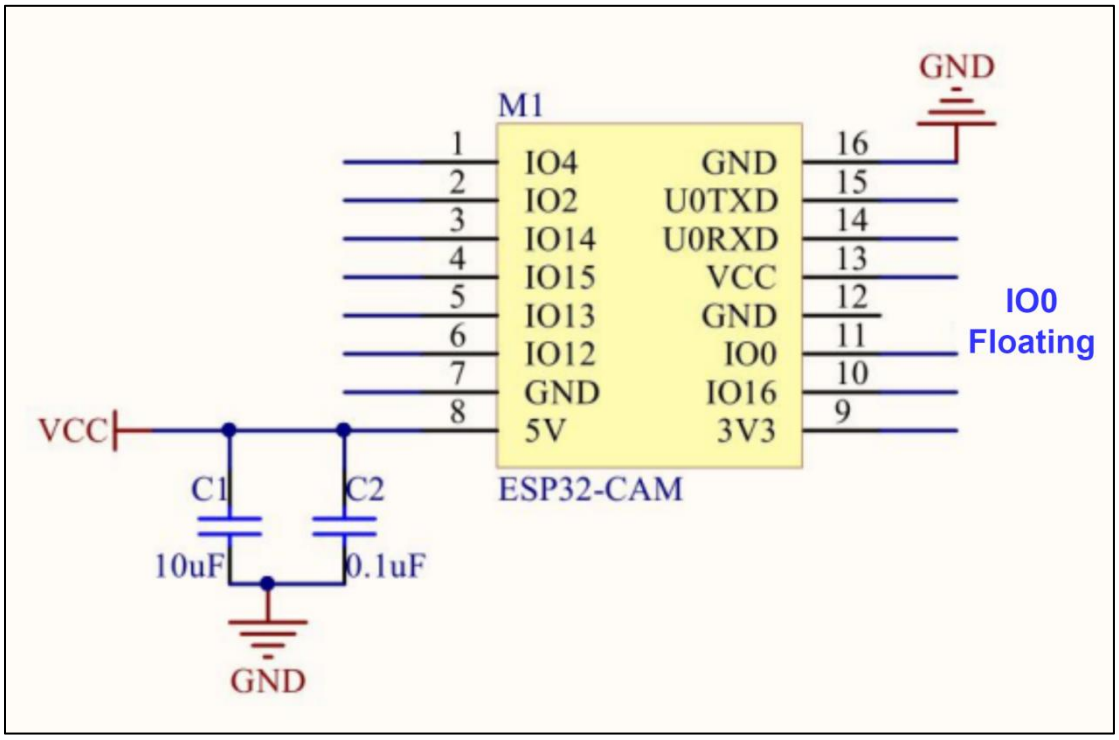
communication.

2.1 Working Principle

ESP32-CAM is equipped with an OV2640 camera module, which transmits the captured image data to the ESP32 chip via a serial port. The ESP32 chip can process the image and transmit it to other devices through I2C communication.

2.2 Schematic Diagram

ESP32-CAM is controlled by the ESP32 chip and is responsible for the control and communication functions of the entire system. The module captures the image data through the OV2640 camera, processes and analyzes it, and finally executes corresponding operations.



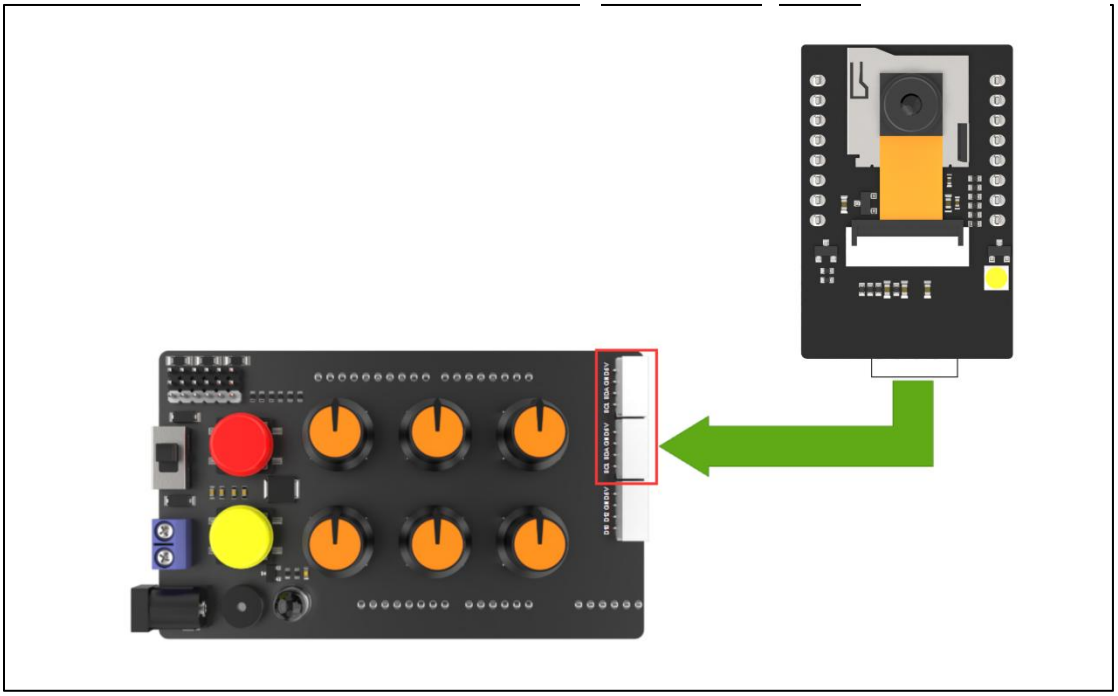
Supply Voltage: 5V

Power Consumption: 6mA-180mA

Working Temperature: -20℃~70℃

2.3 Wiring

Align the pins of ESP32-Cam with the pin headers on the camera expansion board, and connect it to an I2C interface on the MiniArm expansion board with a 4PIN wire.



3. Program Download

Note:

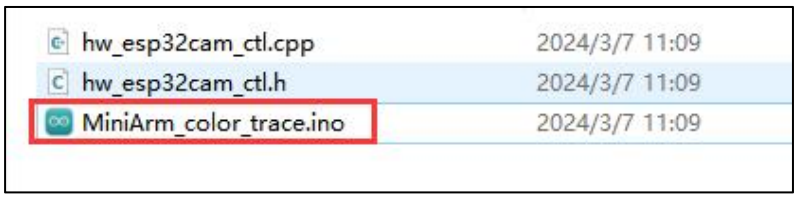
◆ Remove the Bluetooth module before downloading the program. Or the program will fail to download because of the serial port conflict.

◆ Please switch the battery box to the “OFF” when connecting the Type-B download cable. This action prevents the download cable from accidentally touching the power pins of the expansion board, which may cause a short circuit.

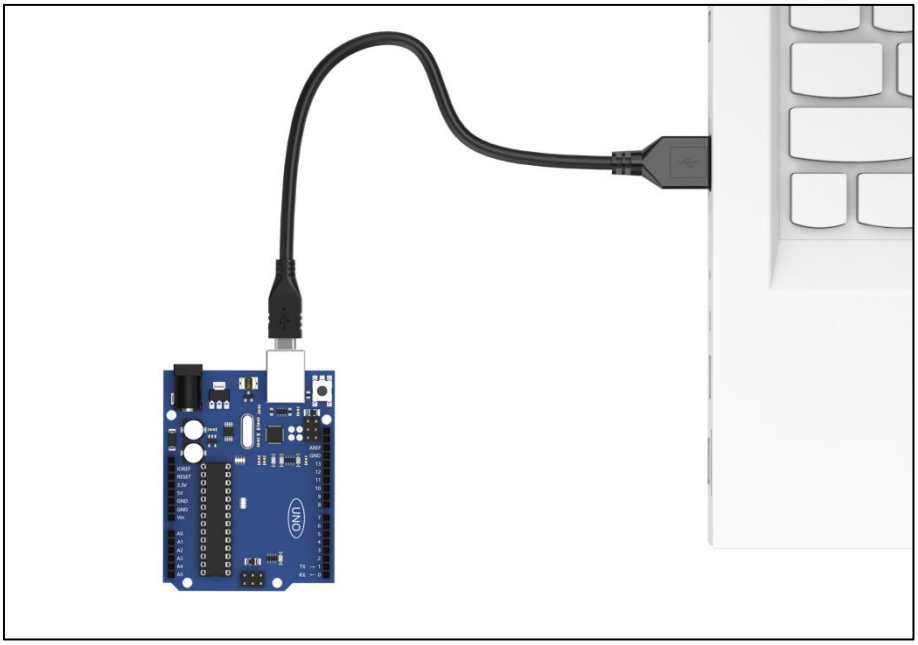
3.1 Arduino UNO Program Download

1) Locate and open the “MiniArm_color_trace\MiniArm_color_trace.ino”

program file in the same directory as this section.



2) Connect Arduino to the computer with the UNO cable (Type-B).



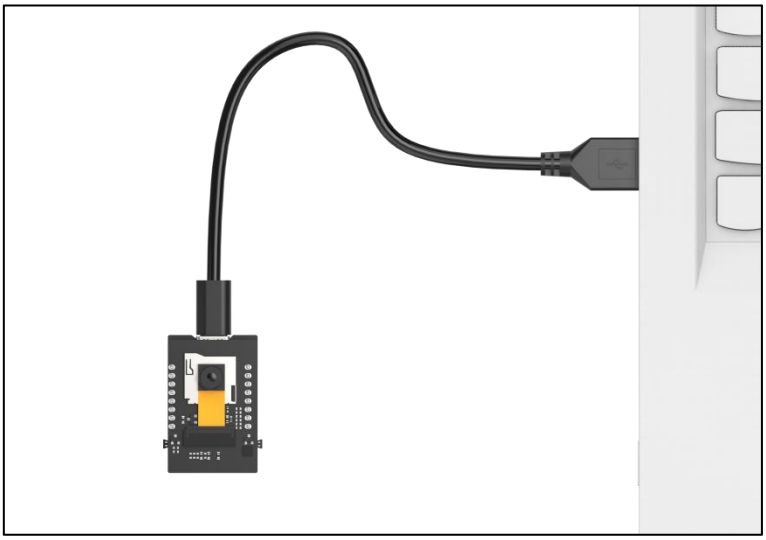
3) Click “Select Development Board”, and the software will automatically detect the current Arduino serial port. Next, click to connect.

4) Click to download the program into Arduino. Then just wait for it to complete.

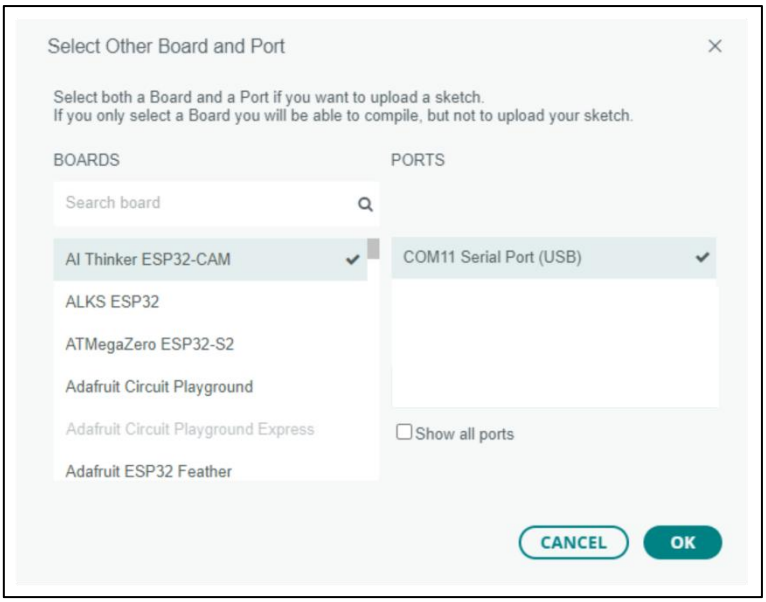
3.2 ESP32-Cam Program Download

1) Locate and open the “ESP32Cam_color_trace\ESP32Cam_color_trace.ino” program file in the same directory as this section.

2) Connect ESP32-Cam to the computer with the USB cable (Type-C).



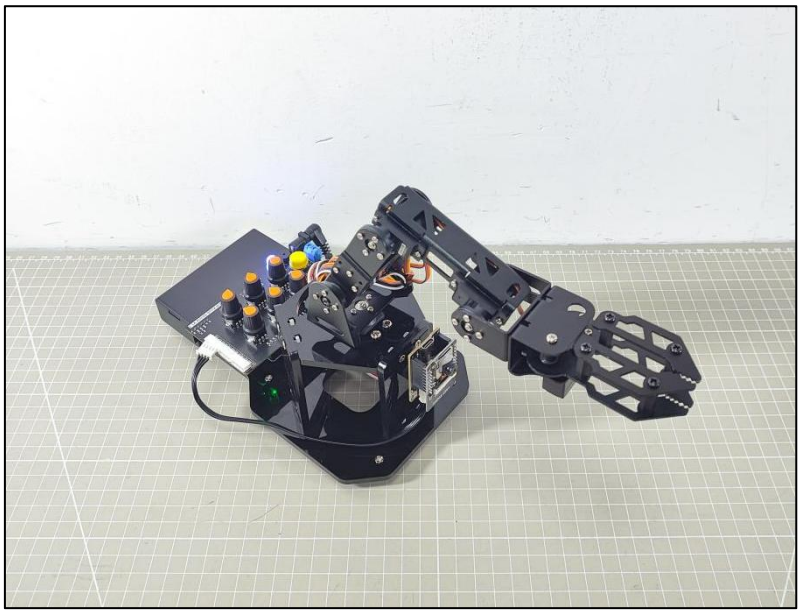
3) Click the “Select Development Board”, then select “AI Thinker ESP32-CAM” and the corresponding port in “Select Other Boards and Ports”.



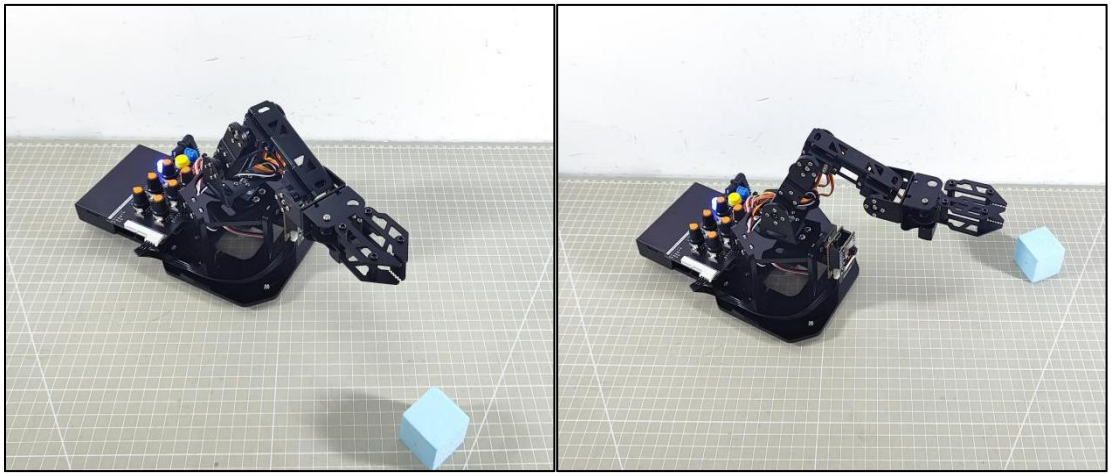
4) Click to download the program into ESP32Cam. Then just wait for it to complete.

4. Program Outcome

1) After the robotic arm is powered on, its pan-tilt returns to the initial position, and the RGB light on the expansion board lights up blue.

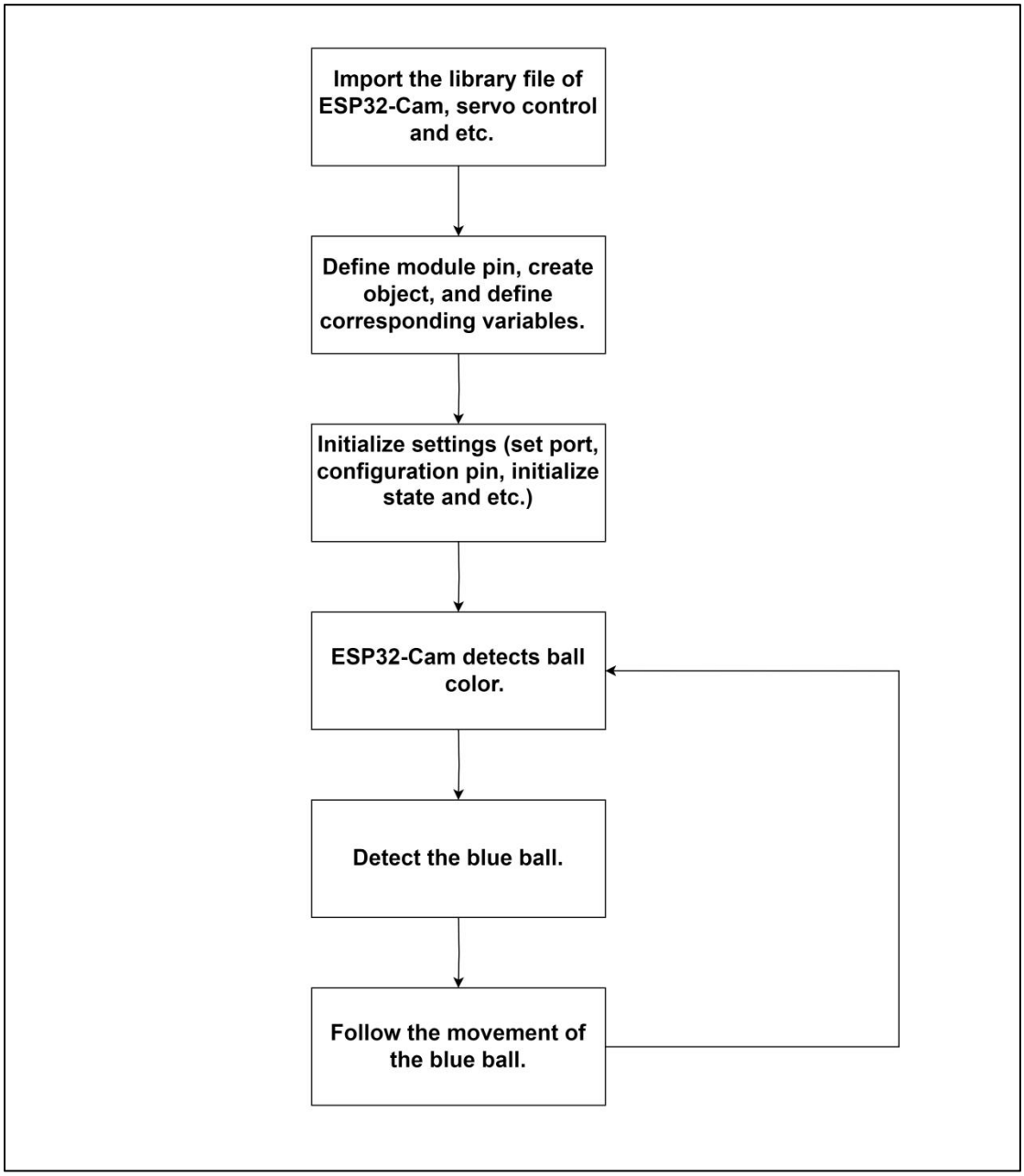


2) When you place the blue block in front of the ESP32-CAM camera and move it left and right, the robotic arm's pan-tilt tracks its movement turning left and right accordingly.



5. Brief Program Analysis

The name of this program is "MiniArm_color_trace.ino". You can refer to the following flowchart for the implementation logic of the program:



5.1 Import Library File

Import the required libraries for RGB control, servo control, buzzer tone, and the ESP32Cam library files for the game.

```
#include <FastLED.h> //import LED library
#include <Servo.h> //import servo library
#include "hw_esp32cam_ctl.h" //import ESP32Cam communication library
#include "tone.h" //tone library
```

5.2 Define Pin and Create Object

1) Firstly, define the buzzer tone and pins, mainly including servo pins, the buzzer pin, and the hardware interface of the RGB light.

```
const static uint8_t servoPins[5] = { 7, 6, 5, 4, 3 };
const static uint8_t buzzerPin = 11;
const static uint8_t rgbPin = 13;
```

2) Then, create the objects for RGB light control, ESP32Cam communication, and servo control.

```
//RGB light color object
static CRGB rgbs[1];
//ESP32Cam communication object
HW_ESP32Cam hw_cam;
//Servo control object
Servo servos[6];
```

3) Next, define the correlation variables controlling servo angles. The array “extended_func_angles[5]” is the desired angles used to uniformly rotate the servos to their designated positions. The “limt_angles[5][2]” is the servo angle limit used to prevent the servos rotating to the physical limit from damaging them. The “servo_angles[5]” is the actual angles of the servos, indicating the current actual angles of the servos.

```
/* The used angle values of secondary development example */
static uint8_t extended_func_angles[5] = { 90,115,180,120,90 };
/* Limit of each joint’s angle */
const uint8_t limt_angles[5][2] = {{0,90},{0,180},{0,180},{25,180},{0,180} };
/* Actual angle values of servo control */
static uint8_t servo_angles[5] = { 90,115,180,120,90 };
static int8_t servo_offset[5];
static uint8_t servo_expect[5];
static uint8_t eeprom_read_buf[16];
```

Regarding the desired and actual values, there may be differences between them during the process of controlling the servo rotation due to various factors. You can check the “servo_control” function called in the “loop” main function.

If you want the servo to gradually move towards the target position, you need to call “servo_angles[i] = servo_angles[i] * 0.9 + extended _func_ a ngles[i] * 0.1” in the control task function.

This allows the servo to move each time at 90% of the actual value and 10% of the expected value, so that the actual value will gradually approach the expected value. And when it is equal to the expected value, the robotic arm stops moving.

```
void servo_control(void) {
  static uint32_t last_tick = 0;
  if (millis() - last_tick < 20) {
    return;
  }
  last_tick = millis();
  for (int i = 0; i < 5; ++i) {
    if(servo_angles[i] > extended_func_angles[i])
    {
      servo_angles[i] = servo_angles[i] * 0.9 + extended_func_angles[i] * 0.1;
      if(servo_angles[i] < extended_func_angles[i])
        servo_angles[i] = extended_func_angles[i];
    }else if(servo_angles[i] < extended_func_angles[i])
    {
      servo_angles[i] = servo_angles[i] * 0.9 + (extended_func_angles[i] * 0.1 + 1);
      if(servo_angles[i] > extended_func_angles[i])
        servo_angles[i] = extended_func_angles[i];
    }

    servo_angles[i] = servo_angles[i] < limt_angles[i][0] ? limt_angles[i][0] : servo_angles[i];
    servo_angles[i] = servo_angles[i] > limt_angles[i][1] ? limt_angles[i][1] : servo_angles[i];
    servos[i].write(i == 0 || i == 5 ? 180 - servo_angles[i] : servo_angles[i]);
  }
}
```

4) Finally, declare the servo’s control function and the ESP32Cam communication task.

```
static void servo_control(void); /* servo control */
void espcam_task(void); /* esp32cam communication task */
static void read_offset(void);
```

5.3 Initialization

1) The “setup ()” function mainly initializes associated hardware devices. Starting with the serial port, which sets the baud rate as “115200” and a read timeout as 500ms.

```
void setup() {
  // put your setup code here, to run once:
  Serial.begin(115200);
  // set the timeout for reading data from the serial port
  Serial.setTimeout(500);
```

2) Assign the servo IO port to facilitate the control through the pin.

```
for (int i = 0; i < 5; ++i) {
  servos[i].attach(servoPins[i]);
}
```

3) Initialize the ESP32Cam communication connector and open the fill light.

```
hw_cam.begin(); //initialize the ESP32Cam communication connector
hw_cam.set_led(10); // control ESP32Cam fill light brightness [brightness value (0~255)]
```

4) Use the “FastLED” library to initialize the RGB lights on the expansion board, and connect them to the RGB pins. Then set the color to blue by “rgbs[0] = CRGB(0, 0, 100)” and use the “FastLED.show” function to display the set color.

```
FastLED.addLeds<WS2812, rgbPin, GRB>(rgbs, 1);
rgbs[0] = CRGB(0, 0, 100);
FastLED.show();
```

5) Set the pin of the buzzer to output mode and stop activating the buzzer

after a short sound.

```
pinMode(buzzerPin, OUTPUT);
tone(buzzerPin, 1000);
delay(100);
noTone(buzzerPin);
```

6) Output the “Start" string through a serial port to indicate that the sensor is ready to start transmitting data.

```
  delay(2000);
  Serial.println("start");
}
```

5.4 Call Subfunction in a Loop

After initialization is complete, the program enters the main “loop” function, where it cyclically calls the “espcam_task” function to communicate with the ESP32Cam. Based on the color recognition results, it performs arm rotation operations. Additionally, it calls the “servo_control” function to control the servos.

```
void loop() {
  // esp32cam communication task
  espcam_task();
  // servo control
  servo_control();
}
```

5.5 ESP32-Cam Communication Task

1) Define the “espcam_task” function. First, two variables are defined. The “last_tick” variable and “millis()” are used for delay operation. Specifically, use the “millis()” function to obtain the running time of the program minus the “last_tick” variable. If the difference is less than 75, the function will return; if

the difference is greater than or equal to 75, it indicates the task has been delayed by 75ms. Then the current time is assigned to the “last_tick” variable for the next delay operation.

The “color_info[4]” array is used to store the return data of the esp32cam. The data corresponding to the subscript is: [0] is the x-coordinate of the upper left corner of the recognized color, [1] is the y-coordinate of the upper left corner of the recognized color, [2] is the width of the recognized color range, and [3] is the height of the recognized color range.

```
void espcam_task(void)
{
  static uint32_t last_tick = 0;
  uint16_t color_info[4];

  // time intervals
  if (millis() - last_tick < 75) {
    return;
  }
  last_tick = millis();
```

2) Next, use the “color_position()” function to determine if the specified color is recognized. If it is recognized, calculate the center point of the color block. Map the position of the center point to the corresponding servo angle and then control the servo’s movement tracking.

```
  if(hw_cam.color_position(color_info)) //if the color is recognized
  {
    uint16_t num = color_info[0] + color_info[2]/2; //calculate the center point of the color block
    uint16_t angle = map(num , 0 , 320 , 60 , 120); //Map to the corresponding servo angle
    extended_func_angles[4] = angle; ///control servo’s movement tracking
  }
}
```

5.6 Servo Control Task

To calculate and set the angle for each servo based on the target angles in the “extended_func_angles” array, the current angles in the “servo_angles”

array, and the angle limits.

```
void servo_control(void) {
  static uint32_t last_tick = 0;
  if (millis() - last_tick < 20) {
    return;
  }
  last_tick = millis();
  for (int i = 0; i < 5; ++i) {
    if(servo_angles[i] > extended_func_angles[i])
```

1) Check the time interval since the last execution. If the interval is less than 20ms, the function will return immediately to avoid updating the servo angle too frequently.

```
void servo_control(void) {
  static uint32_t last_tick = 0;
  if (millis() - last_tick < 20) {
    return;
  }
  last_tick = millis();
```

2) By checking the difference between the “servo_angles[i]” (current servo angles) and the “extended_func_angles[i]” (target angles), adjust the servos’ rotation angles smoothly.

```
for (int i = 0; i < 5; ++i) {
  if(servo_angles[i] > extended_func_angles[i])
  {
    servo_angles[i] = servo_angles[i] * 0.9 + extended_func_angles[i] * 0.1;
    if(servo_angles[i] < extended_func_angles[i])
      servo_angles[i] = extended_func_angles[i];
  }else if(servo_angles[i] < extended_func_angles[i])
  {
    servo_angles[i] = servo_angles[i] * 0.9 + (extended_func_angles[i] * 0.1 + 1);
    if(servo_angles[i] > extended_func_angles[i])
      servo_angles[i] = extended_func_angles[i];
  }
```

◆ Check if the current servo angles are greater than the target angles. If they are, smoothly adjust the angles by taking small steps toward the target angles (each step is 90% of the current angles plus 10% of the target angles).

```
for (int i = 0; i < 5; ++i) {
  if(servo_angles[i] > extended_func_angles[i])
  {
    servo_angles[i] = servo_angles[i] * 0.9 + extended_func_angles[i] * 0.1;
```

◆ If the current servo angles are still greater than the target angles, directly set them to the target angles.

```
if(servo_angles[i] < extended_func_angles[i])
  servo_angles[i] = extended_func_angles[i];
```

◆ Check if the current servo angles are less than the target angles. If they are, smoothly adjust the angles by taking small steps toward the target angles (each step is 90% of the current angles plus 10% of the target angles plus 1).

```
}else if(servo_angles[i] < extended_func_angles[i])
{
  servo_angles[i] = servo_angles[i] * 0.9 + (extended_func_angles[i] * 0.1 + 1);
```

◆ If the current servo angles are still less than the target angles, directly set them to the target angles.

```
  if(servo_angles[i] > extended_func_angles[i])
    servo_angles[i] = extended_func_angles[i];
}
```

3) After adjusting the angles, make sure that the current angles of the servo do not exceed its physical limits.

```
servo_angles[i] = servo_angles[i] < limt_angles[i][0] ? limt_angles[i][0] : servo_angles[i];
servo_angles[i] = servo_angles[i] > limt_angles[i][1] ? limt_angles[i][1] : servo_angles[i];
```

4) Write the calculated angles into the servo.

```
    servos[i].write(i == 0 || i == 5 ? 180 - servo_angles[i] : servo_angles[i]);
  }
}
```

The method “servos[i].write()” is used to set the servo angles. It takes an angle value as a parameter and determines whether to invert it based on the value of “i”.

6. Function Extension

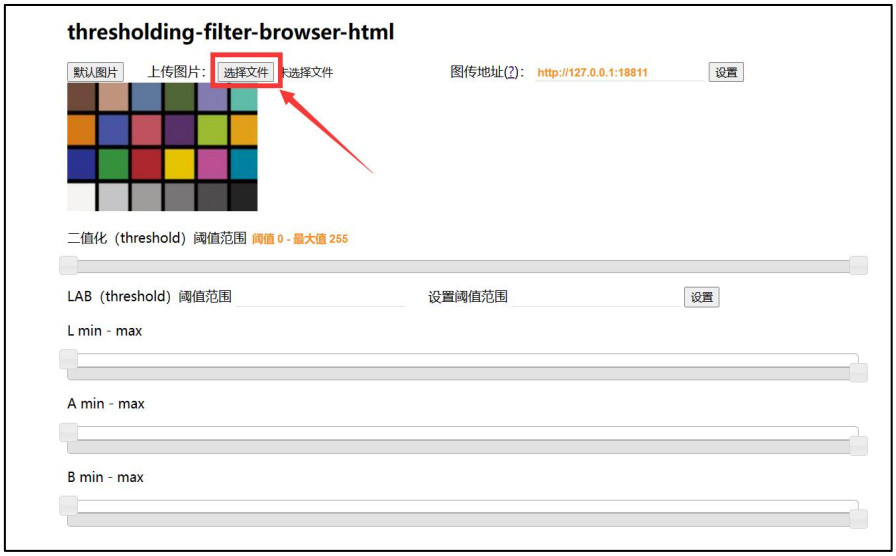
How to modify the recognized color of ESP32-Cam from blue to another color?

Take red as an example, please refer to the following steps:

1) Follow the steps in “6.3 Live Camera Feed” in the same path as this

section to place the red object in a position where the camera can detect it, and save the feedback image.

2) Then, access the “thresholding-filter-browser.html” in the same path as this section, and upload the saved image to the webpage.



3) Drag the slider below to set the LAB threshold range. The white area displayed in the feedback image is recognizable. In this case, we need to set it to recognize the red object. Therefore, set the feedback image to display only the red object.

4) Next, save the LAB threshold value and open “ESP32Cam_color_trace/ESP32Cam_color_trace.ino” in the same path as this section. Comment out the original blue LAB array and replace it with the saved red LAB array. Refer to “3.2 ESP32-Cam Program Download” to flash the modified program into ESP32-Cam.

```
#define COLOR_NUM 2
int8_t color_threshold[COLOR_NUM][6] = {
  // {15,80,15,62,15,50} //, //red {15,80,15,62,15,35}
  // {14, 61, -39, -6, 0, 14},//Green
  // {21,50,-7,8,-35,-11} , //blue
  // {65, 78, -10, -5, 38, 50},//yellow
  // {20, 50, 17, 37, -34, -14}//purple

  // {21, 53, 7, 33, -10, 8} //EVA RED
  {43, 68, -26, -13, 4, 20}, //EVA GREEN
  // {49, 74, -7, 1, -20, -10} //EVA BLUE
  //{43,91,-8,6,-27,-10} //EVA BLUE 2
  {17,81,20,53,10,29} //RED
};
```

5) After the LAB data is modified, refer to “3.1Arduino UNO Program

Download” to flash the program into MiniArm. Once the flashing is complete, the red object can be recognized and tracked.

7. FAQ

Q： since the code was upload, the color can not be recognized.

A: please check that you have connected the 4-pin cable to the I2C interface and turn the knob on the expansion board to the initial position.

Q: why is the color recognized by the camera inaccurate or even incorrect?

A: please try to minimize the complexity of the background. You can use a single color or a simple environment as the background.